Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №5

“ Обробка помилок, файли та серіалізація ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних.

Варіант 10.

Імпорт та експорт складських запасів

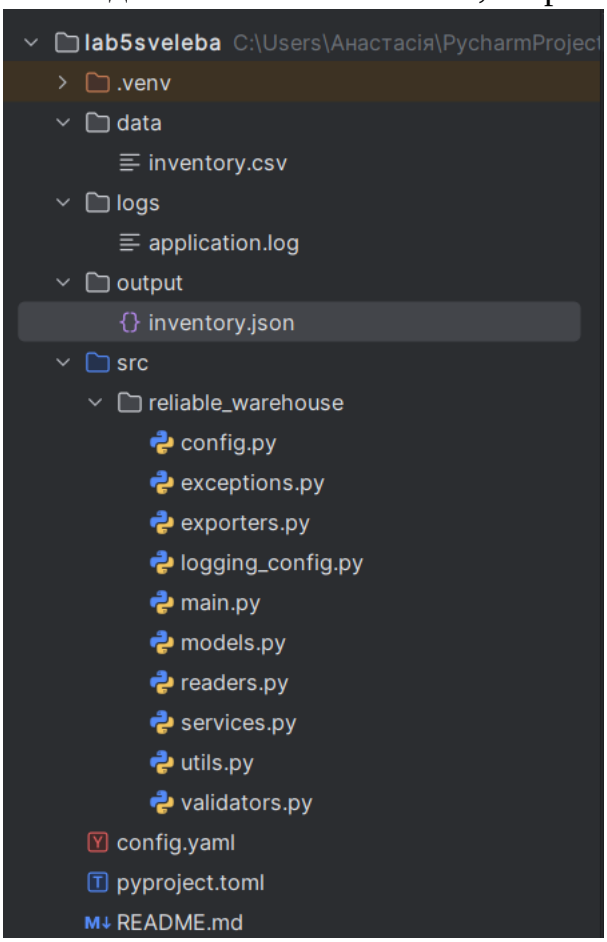
Поля: code, name, quantity, price, category.

Перевіряти code/name, quantity >= 0, price >= 0 та category.

Реалізувати CSV streaming import, low-stock threshold із YAML, JSON export, log warnings для низьких запасів, strict/tolerant mode та atomic export.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/reliable_warehouse зосередила весь програмний код (модулі конфігурації, валідації, читання, експорту, сервісів та власних винятків), залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, config.yaml, README.md), а також допоміжних папок data, output та logs.



2. У файлі pyproject.toml я описала метадані проєкту (reliable-warehouse), його версію, опис, мінімальну версію Python (≥ 3.10) та конфігурацію збірки з використанням setuptools, а також додала залежності від бібліотеки PyYAML.

```toml
[build-system]
requires = ["setuptools>=61.0.0"]
build-backend = "setuptools.build_meta"

[project]
name = "reliable-warehouse"
version = "1.0.0"
description = "Надійний модуль імпорту, експорту та конфігурації складських
запасів (Варіант 10)"
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [
    { name = "Анастасія Най" }
]
classifiers = [
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
]
dependencies = [
    "PyYAML>=6.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=7.0",
    "mypy>=1.0",
]

[tool.setuptools.packages.find]
where = ["src"]

[tool.mypy]
python_version = "3.10"
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = true
```

3. У файлі config.yaml я задала зовнішні параметри для роботи застосунку, зокрема шлях до вхідного CSV-файлу з даними складських запасів та вихідного JSON-файлу, поріг попередження про низькі запаси (low_stock_threshold: 5), режим обробки помилок та налаштування рівня й шляху логування.

```yaml
input:
  path: data/inventory.csv
  format: csv
output:
  path: output/inventory.json
  format: json
processing:
  low_stock_threshold: 5
  skip_invalid: true
logging:
  level: INFO
  path: logs/application.log
```

4. У файлі exceptions.py я реалізувала ієрархію власних доменних винятків (ApplicationError, ConfigurationError, DataImportError, RecordValidationError, DataExportError), а також додала підтримку збереження номери рядка та назви поля для більш точної діагностики помилок валідації.

```
class ApplicationError(Exception):
    """Base application exception."""

class ConfigurationError(ApplicationError):
    """Configuration is invalid."""

class DataError(ApplicationError):
    """Base data exception."""

class DataImportError(DataError):
    """Input data cannot be imported."""

class RecordValidationError(DataError):
    """Record validation failed."""
    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field

class DataExportError(DataError):
    """Output cannot be written."""
```

5. У файлі models.py я створила незмінні dataclass-структури з оптимізацією пам'яті (slots=True, frozen=True) для представлення доменної моделі товару на складі (WarehouseItem) та об'єктів конфігурації програми.

```
from dataclasses import asdict, dataclass
from pathlib import Path

@dataclass(frozen=True, slots=True)
class WarehouseItem:
    code: str
    name: str
    quantity: int
    price: float
    category: str

    def to_dict(self) -> dict[str, object]:
        return asdict(self)

@dataclass(frozen=True, slots=True)
class ProcessingConfig:
    low_stock_threshold: int
    skip_invalid: bool

@dataclass(frozen=True, slots=True)
class LoggingConfig:
    level: str
    path: Path
```

```
@dataclass(frozen=True, slots=True)
class AppConfig:
    input_path: Path
    output_path: Path
    processing: ProcessingConfig
    logging: LoggingConfig
```

6. У файлі config.py я написала функцію безпечного завантаження та парсингу YAML-конфігурації (load_config), додала обробку помилок відсутності чи пошкодження файлу через exception chaining, а також реалізувала перевірку валідності параметрів (validate_config).

```
from pathlib import Path
import yaml
from reliable_warehouse.exceptions import ConfigurationError
from reliable_warehouse.models import AppConfig, LoggingConfig, ProcessingConfig

def load_config(path: Path) -> AppConfig:
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Configuration file not found: {path}") from
error
    except yaml.YAMLError as error:
        raise ConfigurationError("Invalid YAML configuration") from error

    try:
        config = AppConfig(
            input_path=Path(raw["input"]["path"]),
            output_path=Path(raw["output"]["path"]),
            processing=ProcessingConfig(

low_stock_threshold=int(raw["processing"]["low_stock_threshold"]),
                skip_invalid=bool(raw["processing"]["skip_invalid"]),
            ),
            logging=LoggingConfig(
                level=str(raw["logging"]["level"]),
                path=Path(raw["logging"]["path"]),
            ),
        )
    except (KeyError, TypeError, ValueError) as error:
        raise ConfigurationError("Configuration structure is invalid") from
error

    validate_config(config)
    return config

def validate_config(config: AppConfig) -> None:
    if config.processing.low_stock_threshold < 0:
        raise ConfigurationError("low_stock_threshold must be greater than or
equal to 0")
    if not config.input_path.exists():
        raise ConfigurationError(f"Input file not found: {config.input_path}")
```

7. У файлі logging_config.py я налаштувала централізовану систему логування за допомогою модуля logging, визначивши єдиний формат виведення з мітками часу, рівнями важливості, назвами модулів та одночасним записом подій у файл і консоль.

```
import logging
from pathlib import Path
```

```
def configure_logging(level: str, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    numeric_level = getattr(logging, level.upper(), logging.INFO)
    logging.basicConfig(
        level=numeric_level,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
        handlers=[
            logging.FileHandler(path, encoding="utf-8"),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

8. У файлі readers.py я реалізувала потокове читання вхідного CSV-файлу (read_csv_rows) за допомогою генераторів і ітераторів через csv.DictReader, що дозволяє ефективно обробляти великі масиви даних без повного завантаження в оперативну пам'ять.

```
import csv
from collections.abc import Iterator
from pathlib import Path
from reliable_warehouse.exceptions import DataImportError

def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield line_number, row
    except OSError as error:
        raise DataImportError(f"Cannot read input file: {path}") from error
```

9. У файлі validators.py я розробила логіку валідації полів товарів складських запасів (code, name, перевірку невід'ємності кількості й ціни та приналежності категорії до дозволеного списку), генеруючи детальні винятки із зазначенням номерів рядків.

```
from reliable_warehouse.exceptions import RecordValidationError
from reliable_warehouse.models import WarehouseItem

ALLOWED_CATEGORIES = {"electronics", "groceries", "stationery", "hardware",
"clothing"}

def parse_warehouse_item(row: dict[str, str], *, line_number: int) ->
WarehouseItem:
    try:
        code = row.get("code", "").strip()
        if not code:
            raise RecordValidationError("Item code is empty",
line_number=line_number, field="code")

        name = row.get("name", "").strip()
        if not name:
            raise RecordValidationError("Item name is empty",
line_number=line_number, field="name")

        try:
            quantity = int(row["quantity"])
        except (KeyError, TypeError, ValueError) as error:
            raise RecordValidationError("Invalid quantity",
line_number=line_number, field="quantity") from error
```

```
        if quantity < 0:
            raise RecordValidationError("Quantity must be greater than or equal
to 0", line_number=line_number, field="quantity")

        try:
            price = float(row["price"])
        except (KeyError, TypeError, ValueError) as error:
            raise RecordValidationError("Invalid price",
line_number=line_number, field="price") from error

        if price < 0:
            raise RecordValidationError("Price must be greater than or equal to
0", line_number=line_number, field="price")

        category = row.get("category", "").strip().lower()
        if category not in ALLOWED_CATEGORIES:
            raise RecordValidationError(f"Invalid or disallowed category:
'{category}'", line_number=line_number, field="category")

        return WarehouseItem(
            code=code,
            name=name,
            quantity=quantity,
            price=price,
            category=category,
        )
    except KeyError as error:
        raise RecordValidationError(f"Missing required field: {error}",
line_number=line_number) from error
```

10. У файлі services.py я об'єднала бізнес-логіку обробки даних, створивши функцію потокового імпорту та перевірки записів з урахуванням обраної політики обробки помилок (tolerant із пропуском і логуванням або strict) та збиранням статистики успішних, незкоректних і товарів з низьким залишком.

```
import logging
from collections.abc import Iterator
from dataclasses import dataclass
from pathlib import Path
from reliable_warehouse.exceptions import RecordValidationError
from reliable_warehouse.models import WarehouseItem
from reliable_warehouse.readers import read_csv_rows
from reliable_warehouse.validators import parse_warehouse_item

logger = logging.getLogger(__name__)


@dataclass
class ImportStatistics:
    total: int = 0
    valid: int = 0
    invalid: int = 0
    low_stock_count: int = 0


def process_warehouse_items(
        path: Path,
        *,
        skip_invalid: bool,
        low_stock_threshold: int,
        statistics: ImportStatistics,
) -> Iterator[WarehouseItem]:
```

```
    for line_number, row in read_csv_rows(path):
        statistics.total += 1
        try:
            item = parse_warehouse_item(row, line_number=line_number)
        except RecordValidationError as error:
            statistics.invalid += 1
            logger.warning("Invalid record at line %s: %s", error.line_number,
error)
            if skip_invalid:
                continue
            raise

        statistics.valid += 1
        if item.quantity <= low_stock_threshold:
            statistics.low_stock_count += 1
            logger.warning("Low stock warning for item '%s' (Code: %s): quantity
is %s", item.name, item.code,
                           item.quantity)

        yield item
```

11. У файлі exporters.py я реалізувала серіалізацію валідних об'єктів у формат JSON та безпечний атомарний експорт даних (atomic_write через тимчасовий файл і заміну replace()), який гарантує відсутність пошкоджень вихідного файлу у разі аварійного завершення програми.

```
import json
from collections.abc import Iterable
from pathlib import Path
from reliable_warehouse.exceptions import DataExportError
from reliable_warehouse.models import WarehouseItem

def export_items_json(students: Iterable[WarehouseItem], path: Path) -> int:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    count = 0
    try:
        with temporary.open("w", encoding="utf-8") as file:
            file.write("[\n")
            first = True
            for item in students:
                if not first:
                    file.write(",\n")
                json.dump(item.to_dict(), file, ensure_ascii=False, indent=2)
                first = False
                count += 1
            file.write("\n]\n")
        temporary.replace(path)
    except (OSError, TypeError) as error:
        temporary.unlink(missing_ok=True)
        raise DataExportError(f"Cannot export data to {path}") from error
    return count
```

12. У файлі utils.py я створила власний контекстний менеджер logged_operation на базі @contextmanager, який за допомогою perf_counter вимірює час виконання ключових операцій та автоматично логує їх початок, успішне завершення або переривання винятком.

```
from contextlib import contextmanager
import logging
from time import perf_counter
```

```
logger = logging.getLogger(__name__)

@contextmanager
def logged_operation(operation: str):
    logger.info("%s started", operation)
    start = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("%s failed", operation)
        raise
    else:
        elapsed = perf_counter() - start
        logger.info("%s finished in %.6f s", operation, elapsed)
```

13. У файлі main.py я написала головну точку входу в застосунок, яка зв'язує завантаження конфігурації, налаштування логів, запуск потокового конвеєра обробки даних, експорт результатів та обробку критичних помилок із коректним завершенням програми через SystemExit.

```
import logging
from pathlib import Path
from reliable_warehouse.config import load_config
from reliable_warehouse.exceptions import ApplicationError
from reliable_warehouse.exporters import export_items_json
from reliable_warehouse.logging_config import configure_logging
from reliable_warehouse.services import ImportStatistics,
process_warehouse_items
from reliable_warehouse.utils import logged_operation

logger = logging.getLogger(__name__)


def main() -> None:
    try:
        config = load_config(Path("config.yaml"))
        configure_logging(config.logging.level, config.logging.path)

        statistics = ImportStatistics()

        with logged_operation("Warehouse data import/export"):
            items = process_warehouse_items(
                config.input_path,
                skip_invalid=config.processing.skip_invalid,
                low_stock_threshold=config.processing.low_stock_threshold,
                statistics=statistics,
            )
            exported = export_items_json(items, config.output_path)

        logger.info(
            "Total = %s, valid = %s, invalid = %s, low_stock_warnings = %s,
exported = %s",
            statistics.total,
            statistics.valid,
            statistics.invalid,
            statistics.low_stock_count,
            exported,
        )
    except ApplicationError as error:
        logger.error("Application error: %s", error)
        raise SystemExit(1) from error
```

```
if __name__ == "__main__":
    main()
```

14. У файлі inventory.csv (у папці data) я сформувала вхідний тестовий набір даних складських запасів, який містить як коректні записи, так і спеціально створені помилки для перевірки надійності системи (порожні поля, від'ємну кількість, некоректні ціни, недозволені категорії та товари з низьким залишком для тригерів попереджень).

```
code,name,quantity,price,category
E101,Laptop,12,799.99,electronics
G202,Coffee,3,14.50,groceries
S303,Notebook,50,2.50,stationery
H404,Hammer,,15.00,hardware
,Printer,5,199.99,electronics
F505,Apple,-5,1.20,groceries
E106,Mouse,2,25.00,unknown_cat
```

15. У файлі README.md я створила вичерпну технічну документацію проєкту, описавши структуру полів варіанта, інструкції зі встановлення залежностей та запуску програми, а також ключові особливості реалізованого надійного пайплайну обробки даних.

```
# Reliable Warehouse Importer & Exporter (Варіант 10)

Надійний Python-модуль для потокового імпортування, валідації та експорту даних складських запасів із підтримкою гнучкої конфігурації, обробки винятків та атомарного запису.

## Особливості реалізації
- **Потокове читання (Streaming I/O):** Ефективна обробка великих CSV-файлів за допомогою генераторів без завантаження всього обсягу в оперативну пам'ять (RAM).
- **Ієрархія винятків та Chaining:** Власні доменні винятки (`ApplicationError`, `DataImportError`, `RecordValidationError`, `ConfigurationError`) із збереженням першопричини через `raise ... from error`.
- **Політики обробки помилок:**
  - *Tolerant Mode:* Пропуск некоректних рядків з їх логуванням і продовження обробки.
  - *Strict Mode:* Миттєве переривання процесу при першій же помилці валідації.
- **Атомарний експорт (Atomic Write):** Запис результатів спочатку у тимчасовий файл (`.tmp`) з подальшою атомарною заміною оригінального файлу, що захищає дані від пошкодження у разі аварійного завершення програми.
- **Контекстні менеджери:** Безпечне управління ресурсами файлів та вимірювання часу виконання операцій.
- **Логування:** Детальна діагностика подій та попереджень про низькі запаси товарів (`low_stock_threshold`) за допомогою вбудованого модуля `logging`.

---

## Структура полів (Варіант 10)
Вхідний файл (`data/inventory.csv`) містить такі поля:
1. `code` — унікальний код товару (обов'язковий рядок).
2. `name` — назва товару (обов'язковий рядок).
3. `quantity` — кількість на складі (ціле число $\ge 0$).
4. `price` — ціна за одиницю (число з плаваючою комою $\ge 0$).
5. `category` — категорія товару (має належати до списку дозволених: `electronics`, `groceries`, `stationery`, `hardware`, `clothing`).

---

## Інструкція з запуску
```

```
1. **Клонування репозиторію та встановлення залежностей:**
   ```bash
   python -m pip install --upgrade pip
   pip install -e .

2. **Запуск застосунку:**
   ```bash
   python -m reliable_warehouse.main
```

Результати:

```
C:\Users\Анастасія\PycharmProjects\lab5sveleba\.venv\Scripts\python.exe C:\Users\Анастасія\PycharmProjects\lab5sveleba\src\reliable_wareho
2026-10-13 22:42:59,484 | INFO | reliable_warehouse.utils | Warehouse data import/export started
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Low stock warning for item 'Coffee' (Code: G202): quantity is 3
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record at line 5: Invalid quantity
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record at line 6: Item code is empty
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record at line 7: Quantity must be greater than or equal to 0
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record at line 8: Invalid or disallowed category: 'unknown_cat'
2026-10-13 22:42:59,486 | INFO | reliable_warehouse.utils | Warehouse data import/export finished in 0.001643 s
2026-10-13 22:42:59,486 | INFO | __main__ | Total = 7, valid = 3, invalid = 4, low_stock_warnings = 1, exported = 3

Process finished with exit code 0
```

- У файлі application.log (у папці logs) автоматично сформувався детальний журнал виконання програми, де зафіксовано часові мітки, рівні важливості (INFO, WARNING), назви модулів, успішні етапи імпорту й експорту, а також попередження про товари з низьким залишком на складі та рядки з помилками валідації.

```
2026-10-13 22:42:59,484 | INFO | reliable_warehouse.utils | Warehouse data
import/export started
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Low stock
warning for item 'Coffee' (Code: G202): quantity is 3
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record
at line 5: Invalid quantity
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record
at line 6: Item code is empty
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record
at line 7: Quantity must be greater than or equal to 0
2026-10-13 22:42:59,485 | WARNING | reliable_warehouse.services | Invalid record
at line 8: Invalid or disallowed category: 'unknown_cat'
2026-10-13 22:42:59,486 | INFO | reliable_warehouse.utils | Warehouse data
import/export finished in 0.001643 s
2026-10-13 22:42:59,486 | INFO | __main__ | Total = 7, valid = 3, invalid = 4,
low_stock_warnings = 1, exported = 3
```

- У файлі inventory.json (у папці output) успішно збережено структурований результат обробки даних у форматі JSON, який містить виключно валідні записи товарів складських запасів, сформовані за допомогою потокового запису та атомарного експорту через тимчасовий файл.


```
[
{
  "code": "E101",
  "name": "Laptop",
  "quantity": 12,
  "price": 799.99,
  "category": "electronics"
},
{
  "code": "G202",
```

```
  "name": "Coffee",
  "quantity": 3,
  "price": 14.5,
  "category": "groceries"
},
{
  "code": "S303",
  "name": "Notebook",
  "quantity": 50,
  "price": 2.5,
  "category": "stationery"
}
]
```


Висновок: у ході лабораторної роботи я розробила надійний модуль імпорту, валідації та експорту складських запасів (Варіант 10). Я успішно застосувала доменну ієрархію винятків із exception chaining, контекстні менеджери, потокове читання даних (streaming I/O) та атомарний експорт через тимчасові файли. Гнучке налаштування через YAML-конфігурацію, підтримка strict/tolerant режимів і детальне логування забезпечили стабільну та безпечну роботу програми в умовах непередбачуваних помилок вхідних даних. Отримані результати підтвердили готовність створеного програмного компонента до використання у професійних Python-проєктах.